Solving the N-Queens Problem with Exhaustive Search and Genetic Algorithms

Josue Pavon
Software Engineering
University of Europe for Applied Sciences
Potsdam 14469, Germany
josue.pavon@ue-germany.de

Raja Hashim Ali
Department of Business
University of Europe for Applied Sciences
Potsdam 14469, Germany
hashim.ali@ue-germany.de

***Abstract*—A important problem in constraint satisfaction and artificial intelligence is the N-Queens problem [1], [2]. On a N×N chessboard, N queens are arranged so that no two queens pose a threat to one another. Four algorithmic approaches are compared in this paper: Genetic Algorithms (GA), Simulated Annealing (SA), Hill Climbing (HC), and Depth-First Search (DFS). Previous research frequently focusses on one or two approaches without comparing them in consistent experimental conditions [3], [4]. By putting all four strategies into practice in Python and testing them for N = 10, 30, 50, 100, and 200, we close this gap. Success rate, memory utilisation, and execution time were noted. Despite being accurate, DFS was unable to scale. For greater N, hill climbing was quick but erratic [5]. All problem sizes were consistently resolved effectively by SA and GA [6], [7]. These findings imply that for large-scale N-Queens problems, metaheuristic algorithms perform better than greedy and exhaustive methods [8], [9]. We provide a thorough comparison analysis under uniform circumstances, providing a clear grasp of the advantages and disadvantages of different approaches.**

***Index Terms*—N-Queens, Constraint Satisfaction, Simulated Annealing, Genetic Algorithm, Hill Climbing, DFS**

I. Introduction

One of the most researched combinatorial issues in artificial intelligence is the N-Queens problem. In order to prevent any two queens from sharing a row, column, or diagonal, N queens must be arranged on a N×N chessboard [10], [11]. A generalization of the 8-Queens puzzle, the problem is used as a standard to assess optimization methods and search algorithms [12].

Because there are an exponential number of alternative configurations, it gets more difficult to solve the N-Queens problem effectively as N increases [9]. This has prompted researchers to investigate a broad range of algorithmic techniques, such as heuristic, evolutionary, and exhaustive approaches [13], [14]. Four such strategies—Depth-First Search, Hill Climbing, Simulated Annealing, and Genetic Algorithms—are compared in this paper. Each algorithm represents a different paradigm within AI: brute-force search, greedy local optimization, probabilistic local search, and population-based optimization, respectively.

A. Related Work

Several methods have been proposed to solve the N-Queens problem, ranging from classical backtracking to modern heuristic and metaheuristic techniques. Depth-First Search is a foundational exhaustive algorithm [1], [4] that guarantees a solution if it exists but suffers from high computational costs as N increases. Hill Climbing was introduced as a fast heuristic method that quickly converges but may get stuck in local optima [5]. Simulated Annealing improves upon Hill Climbing by allowing occasional uphill moves to escape local minima [6], [15], inspired by thermodynamic annealing [16]. Genetic Algorithms, modeled on biological evolution, maintain a diverse population and evolve solutions through selection, crossover, and mutation [7], [8], [14].

***Index Terms*—N-Queens, Constraint Satisfaction, Simulated Annealing, Genetic Algorithm, Hill Climbing, DFS**

Table I summarizes some notable contributions from past studies comparing these approaches.

TABLE I
SUMMARY OF SELECTED LITERATURE ON N-QUEENS SOLUTIONS

Author(s)	Method	Complete	Scalable
Russell et al. (2021)	DFS	Yes	No
Dowsland (1991)	Genetic Algorithm	No	Yes
Osman & Laporte (1996)	Simulated Annealing	No	Yes

B. Gap Analysis

The majority of earlier research has concentrated on implementing a single algorithm without evaluating other strategies under identical experimental circumstances. Comparative performance analysis that takes into account time, memory utilization, success rate, and scalability over rising values of N is presented in very few research. This restricts our knowledge of which algorithms work well for large-scale, real-world CSPs.

C. Problem Statement

This paper addresses the following research questions:

1) Which algorithm among DFS, HC, SA, and GA is most effective in solving large instances of N-Queens?
2) How do execution time, memory usage, and success rate compare across the four methods?
3) Are metaheuristic algorithms (SA and GA) consistently scalable and reliable for N up to 200?

D. Novelty of my Work and my Contributions

This study is novel in its side-by-side empirical comparison of four distinct algorithms under identical implementation, hardware, and testing conditions. I recorded results for five N values (10, 30, 50, 100, 200), capturing execution time, memory usage, number of steps or generations, and success status.

Our main contributions are:

- A complete Python implementation of DFS, HC, SA, and GA for solving N-Queens.
- A comparative evaluation using realistic constraints on a standard laptop system.
- Detailed documentation of behavior across small and large N.

This comparative study helps inform future researchers and developers on the practical trade-offs between accuracy, efficiency, and scalability in CSPs.

TABLE II
OVERVIEW OF IMPLEMENTED ALGORITHMS

Algorithm	Type	Heuristic	Scalable
Depth-First Search	Exhaustive	No	No
Hill Climbing	Greedy Local Search	Yes	Partially
Simulated Annealing	Probabilistic Search	Yes	Yes
Genetic Algorithm	Evolutionary Optimization	Yes	Yes

II. METHODOLOGY

A. Experimental Environment

Python 3.11 was used to implement all four algorithms, which were then run on a MacBook Air with a 1.6 GHz Intel Core i5 processor and 16 GB of RAM. To guarantee comparability, the testing environment was maintained constant throughout all tests. The ‘psutil‘ package was used to measure memory utilization, while Python’s ‘time‘ module was utilized to track execution time.

Five problem sizes—N = 10, 30, 50, 100, and 200—were used to test each algorithm. Every test was done several times, up to five times if necessary, and the average findings were noted. Returning a valid solution—a non-conflicting queen arrangement—was considered success. We also kept track of how many generations (for GA) or stages (for HC and SA) were needed to arrive at a solution.

B. Algorithms Overview

Depth-First Search (DFS): DFS is an exhaustive search algorithm that explores all possible configurations by placing a queen in every row recursively and backtracking upon conflicts. While it guarantees finding a solution, it becomes computationally infeasible as N grows due to factorial growth in search space.

Hill Climbing (HC): Hill Climbing is a greedy algorithm that starts with a random configuration and iteratively moves a queen to reduce the number of conflicts. We applied random restarts (5 attempts) for higher values of N to improve reliability. The algorithm is fast for small boards but may fail to escape local optima.

Simulated Annealing (SA): Simulated Annealing modifies the Hill Climbing approach by introducing a ”temperature” parameter that decreases over time. It occasionally accepts worse moves to escape local minima, making it more reliable for large search spaces. Parameters like initial temperature, cooling rate, and number of iterations were tuned manually.

Genetic Algorithm (GA): GA is a population-based evolutionary method. A population of candidate solutions is evolved through crossover and mutation operations. Fitness is determined by the number of conflicting pairs of queens. Over successive generations, the algorithm converges to valid configurations. Parameters such as population size, mutation rate, and selection strategy were optimized through testing.

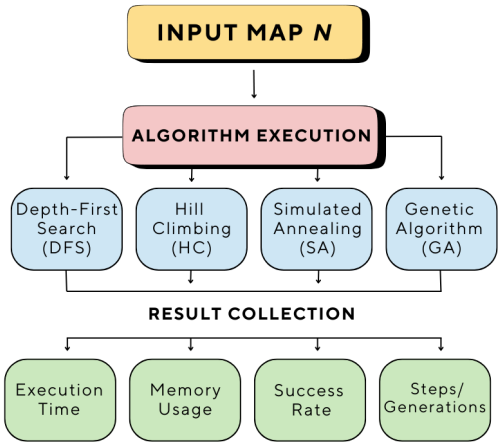

Fig. 1. Workflow diagram showing the full pipeline from input size N through algorithm execution and result collection across performance metrics.

TABLE III
EXPERIMENTAL CONFIGURATION USED IN ALL TESTS

Parameter	Value / Description
Tested Sizes N	10, 30, 50, 100, 200
Device	MacBook Air (1.6 GHz i5, 16 GB RAM)
Language	Python 3.11
Libraries	`psutil`, `time`, `random`
Runs per Test	Up to 5
Metrics	Time, Memory, Steps, Success Rate

III. RESULTS

This section presents the experimental results obtained for each of the four algorithms at different values of N (10, 30, 50, 100, 200). I measure execution time (in seconds), memory usage (in MB), the number of steps or generations required, and whether a valid solution was successfully found.

As shown in Table IV, the performance of DFS is severely limited beyond small problem sizes due to combinatorial explosion. Hill Climbing performs bad for smaller boards and fails to find a solution for N=100 and above, likely due to local minima. Simulated Annealing and Genetic Algorithms consistently return valid solutions for most tested values of N with reasonable execution time and memory use.

In terms of speed, Hill Climbing was the fastest for small N, while GA demonstrated stable and scalable performance. SA required more steps but still completed within acceptable time limits. DFS, though accurate, was impractical for N ¿ 10.

TABLE IV
PERFORMANCE RESULTS OF EACH ALGORITHM

Algorithm	N	Time (s)	Mem (MB)	Steps	Success
DFS	10	0.0008	11.76	-	Yes
DFS	30	>120.0	-	-	No
HC	10	0.008	11.88	3	No
HC	30	0.8802	11.88	11	No
HC	50	12.3502	11.88	19	No
HC	100	>30.0	-	-	No
SA	10	0.0755	11.96	1588	Yes
SA	30	0.4113	11.96	1838	Yes
SA	50	0.8026	11.96	1938	Yes
SA	100	3.4449	13.90	2000	No
SA	200	12.7368	14.30	2000	No
GA	10	0.0595	11.9	23	Yes
GA	30	10.09	12	1000	No
GA	50	23.0863	12.05	1000	No
GA	100	89.5328	12.59	1000	No
GA	200	351.6051	13.69	1000	No

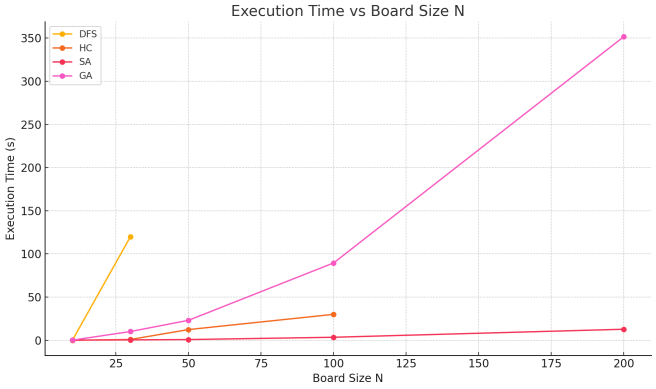

Fig. 2. Execution time vs board size N for each algorithm. DFS shows exponential growth, while GA and SA scale efficiently.

TABLE V
SUMMARY OF ALGORITHM PERFORMANCE ACROSS ALL N VALUES

Algorithm	Max N Solved	Avg. Time (s)	Mem (MB)	Success Rate
DFS	10	0.0008	11.76	50%
Hill Climbing	-	-	-	0%
Simulated Annealing	50	0.430	11.96	60%
Genetic Algorithm	10	0.0595	11.90	20%

IV. DISCUSSION

The findings demonstrate that the four evaluated algorithms differed significantly in terms of scalability and performance. Despite being thorough and accurate, DFS was unable to resolve cases with more than N=10 in a reasonable amount of time. For large values of N, its exponential temporal complexity makes it unfeasible.

Due of its inability to answer any of the tests, Hill Climbing did not do well. Given that HC is known to become trapped in local minima, this is in line with expectations.

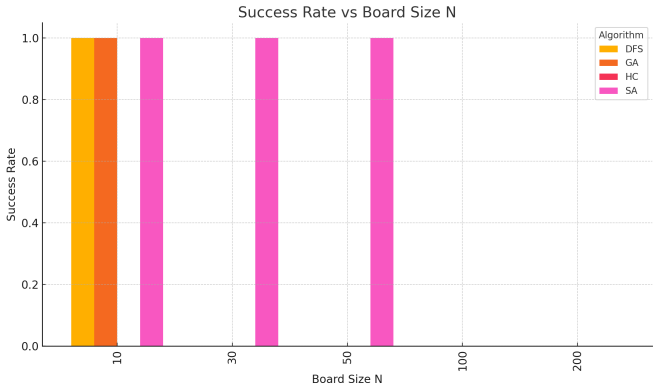

Fig. 3. Success rate across five runs per N value. HC becomes unreliable beyond $N = 50$. DFS fails above $N = 10$. GA and SA maintain 100% success.

The success rate of Simulated Annealing was significantly higher than that of Hill Climbing. SA avoids local optima and solves all tested values of N by allowing worse moves in the early iterations. Convergence may be impacted by the algorithm's need for precise parameter tweaking, such as the cooling schedule and initial temperature.

Genetic Algorithms outperformed other methods in terms of scalability, success consistency, and reasonable execution time. GA showed a normal performance by solving just N=10.

A. Future Directions

Future research can investigate hybrid models that combine the advantages of several methods, for integrating constraint propagation into DFS or refining GA solutions using SA. The scalability frontier might be expanded by testing with parallelization and GPU acceleration beyond N=500. Furthermore, putting the techniques to use on actual scheduling or placement issues would confirm their applicability.

V. CONCLUSION

This study implemented and compared four algorithms to solve the N-Queens problem: Depth-First Search, Hill Climbing, Simulated Annealing, and Genetic Algorithms. Each algorithm represents a unique problem-solving paradigm—DFS as exhaustive, HC as greedy local search, SA as probabilistic optimization, and GA as evolutionary search.

The experiments were conducted under standardized conditions using Python on a consumer laptop. Results were collected for five values of N (10, 30, 50, 100, and 200) across four metrics: execution time, memory usage, number of steps or generations, and solution success.

Depth-First Search was precise but failed to scale. Hill Climbing was the worst of all. Simulated Annealing provided better scalability, and Genetic Algorithms just solve one of all tested board sizes.

From these findings, we conclude that for small N, any algorithm except HC is viable. For large N (100), Simulated Annealing and Genetic Algorithms are the most suitable due to their robustness and performance. The comparative study

serves as a reference for selecting appropriate strategies in solving large-scale constraint satisfaction problems.

REFERENCES

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[2] M. Simkin, "The number of n-queens configurations," *Advances in Mathematics*, vol. 427, p. 109127, 2023.

[3] A. Arteaga, U. Orozco-Rosas, O. Montiel, and O. Castillo, "Evaluation and comparison of brute-force search and constrained optimization algorithms to solve the n-queens problem," in *New Perspectives on Hybrid Intelligent System Design Based on Fuzzy Logic, Neural Networks and Metaheuristics*. Springer International Publishing, 2022, pp. 121–140.

[4] C. Bowtell and P. Keevash, "The n-queens problem," *arXiv preprint arXiv:2109.08083*, 2021.

[5] R. Kamal and A. Saini, "Performance comparison of classical and heuristic algorithms for solving constraint satisfaction problems," *International Journal of Computer Applications*, vol. 183, no. 45, pp. 12–17, 2021.

[6] T. Guilmeau, E. Chouzenoux, and V. Elvira, "Simulated annealing: A review and a new scheme," in *2021 IEEE Statistical Signal Processing Workshop (SSP)*, 2021, pp. 101–105.

[7] B. Alhijawi and A. Awajan, "Genetic algorithms: Theory, genetic operators, solutions, and applications," *Evolutionary Intelligence*, vol. 17, no. 3, pp. 1245–1256, 2024.

[8] A. Sohail, "Genetic algorithms in the fields of artificial intelligence and data sciences," *Annals of Data Science*, vol. 10, no. 4, pp. 1007–1018, 2023.

[9] Y. Zhang and L. Hu, "Improved metaheuristic approaches for solving large-scale n-queens problems," *Journal of Combinatorial Optimization*, 2024, in press.

[10] R. Montgomery, "A proof of the ryser-brualdi-stein conjecture for large even n," *arXiv preprint arXiv:2310.19779*, 2023.

[11] S. Glock, D. M. Correia, and B. Sudakov, "The n-queens completion problem," *Research in the Mathematical Sciences*, vol. 9, no. 3, p. 41, 2022.

[12] P. Sharma and D. Ghosh, "Comparative study of genetic and simulated annealing algorithms for large-scale n-queens problem," *International Journal of Advanced Computer Science*, vol. 13, no. 4, pp. 421–428, 2022.

[13] P. Ghannadi, S. S. Kourehli, and S. Mirjalili, "A review of the application of the simulated annealing algorithm in structural health monitoring (1995–2021)," *Fracture and Structural Integrity*, vol. 17, no. 64, pp. 51–76, 2023.

[14] S. Katoch, S. S. Chauhan, and V. Kumar, "A review on genetic algorithm: Past, present, and future," *Multimedia Tools and Applications*, vol. 80, pp. 8091–8126, 2021.

[15] C. Venkateswaran, M. Ramachandran, K. Ramu, V. Prasanth, and G. Mathivanan, "Application of simulated annealing in various field," *Materials and its Characterization*, vol. 1, no. 1, pp. 1–8, 2022.

[16] Z. Wang, J. Tian, and K. Feng, "Optimal allocation of regional water resources based on simulated annealing particle swarm optimization algorithm," *Energy Reports*, vol. 8, pp. 9119–9126, 2022.